

ESTRUTURAS DE DADOS E ALGORITMOS

Pilhas, Filas e Filas Duplas

Aula 7 · Estruturas de dados avançadas, parte 1

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Inserir no início: é mais barato numa ArrayList ou numa LinkedList? (Aula 1)

2

O que acontece em Java a uma recursão sem caso base? (Aula 4)

3

Qual o resultado de fundir 2 3 8 9 com 1 4 5 7? (Aula 4)

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Três situações do dia a dia. Quem sai primeiro?

Ctrl + Z

Carrega três vezes. Que ações são desfeitas, e por que ordem?

A última a entrar é a primeira a sair.

PILHA

A fila da cantina

Quem é servido primeiro?

O primeiro a chegar é o primeiro a sair.

FILA

Um baralho de cartas

Pode tirar ou pôr cartas por cima e por baixo.

Entra e sai pelas duas pontas.

FILA DUPLA

Três regras simples de entrada e saída. Hoje damos-lhes nome e implementamo-las.

Por onde entra, por onde sai

1

Pilhas

LIFO: o último a entrar é o primeiro a sair.

2

Filas

FIFO: o primeiro a entrar é o primeiro a sair.

3

Filas duplas

Entrar e sair pelas duas pontas.

4

Em Java

As classes prontas e os exercícios.

Pergunta-guia: como é que uma simples regra de acesso torna uma estrutura tão útil?

1

PARTE 1

Pilhas

Last In, First Out.

O último a entrar é o primeiro a sair

LIFO Last In, First Out

Só se acede ao elemento do topo: o mais recente.

push(x)

Coloca x no topo.

pop()

Remove e devolve o elemento do topo (o mais recente).

peek()

Espreita o topo, sem o remover.

size() · isEmpty()

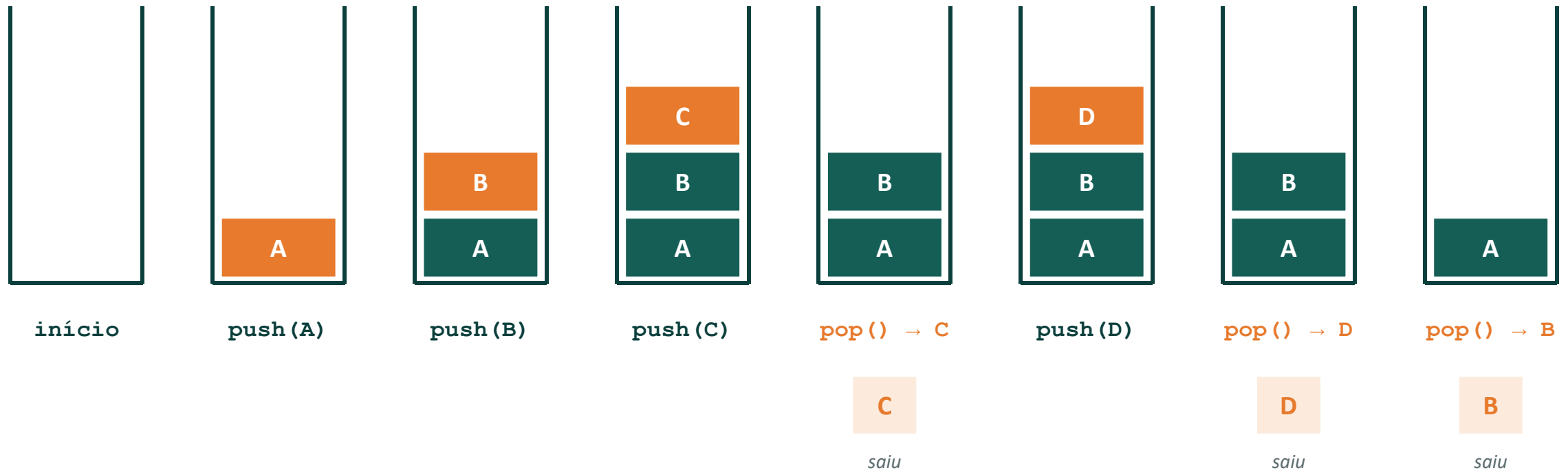
Quantos elementos há? Está vazia?

Uma pilha de pratos



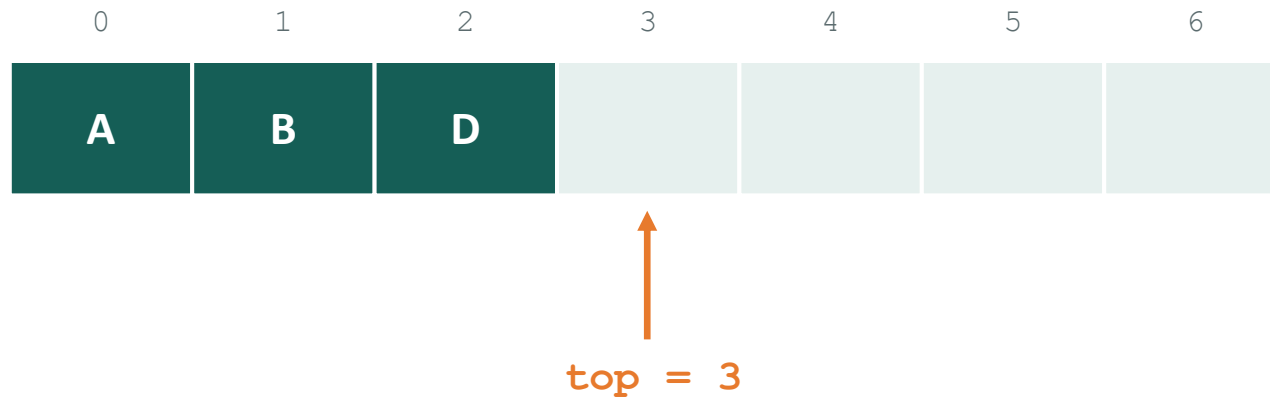
põe-se e tira-se sempre por cima

Sete operações, passo a passo



Repare: sai sempre o que está em cima — o mais recente.

Uma pilha com um array



próxima posição livre = número de elementos

push(x) :

```
se top == capacidade: erro (cheia)
A[top] ← x;  top ← top + 1
```

pop() :

```
se top == 0: erro (vazia)
top ← top - 1;  devolver A[top]
```

Custo

push **$O(1)$**

pop **$O(1)$**

peek **$O(1)$**

size **$O(1)$**

Só se mexe no topo: nunca é preciso deslocar elementos.

VERIFICAÇÃO RÁPIDA

Numa pilha vazia faz-se:

`push(1)` `push(2)` `pop()` `push(3)` `push(4)` `pop()` `pop()`

Que valores saem, e por que ordem? O que fica na pilha?

RESPOSTA

Saem: **2, 4, 3**

Fica: **1**

Desenhe a pilha em cada passo: é a forma mais segura de não se enganar.

Onde encontramos pilhas

1

Anular (Ctrl + Z)

Cada ação é empilhada; anular desempilha a mais recente.

2

Chamadas de funções

A "call stack" guarda as chamadas em curso. Recursão sem fim
→ `StackOverflowError`.

3

Parênteses equilibrados

([{ }]): empilha ao abrir, desempilha ao fechar e confirma o par.

4

Botão "voltar" do browser

As páginas visitadas ficam numa pilha; voltar desempilha a mais recente.

2

PARTE 2

Filas

First In, First Out.

O primeiro a entrar é o primeiro a sair

FIFO First In, First Out

Entra-se atrás; sai-se à frente: o mais antigo.

enqueue(x)

Coloca x no fim da fila.

dequeue()

Remove e devolve o elemento da frente (o mais antigo).

front()

Espreita a frente, sem remover.

size() · isEmpty()

Quantos elementos há? Está vazia?

A fila da cantina



quem chega primeiro é servido primeiro

Seis operações, passo a passo

início

vazia

enqueue (A)



enqueue (B)



enqueue (C)



dequeue () → A



A

saiu pela frente

enqueue (D)



dequeue () → B



B

saiu pela frente

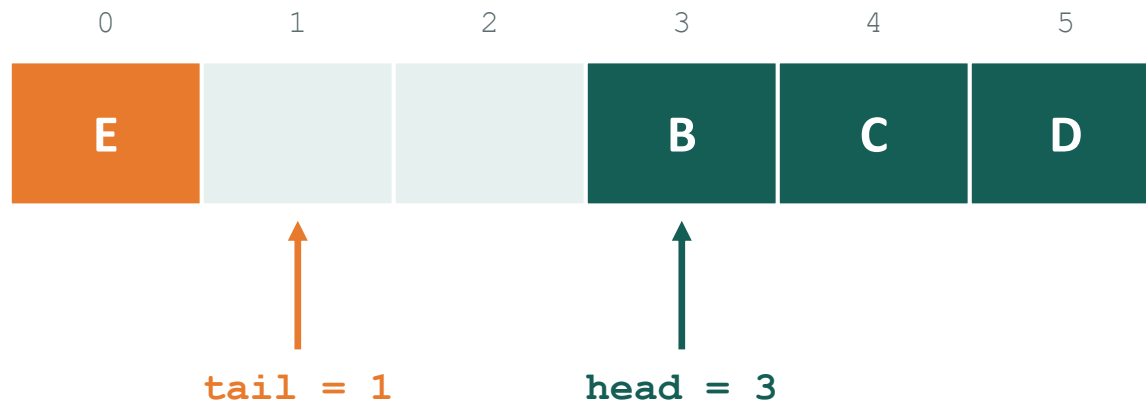
Compare

Com a mesma sequência numa pilha, o primeiro a sair seria C. Na fila é A: o mais antigo.

frente ← ... ← fim

Uma fila com um array circular

Quando chega ao fim do array, o índice dá a volta ao início:



Fila: B C D E (E entrou na posição 0 depois de dar a volta)

enqueue (x) :

```
A[tail] ← x; tail ← (tail + 1) % cap; count++
```

dequeue () :

```
x ← A[head]; head ← (head + 1) % cap; count--  
devolver x
```

O truque do módulo

Com capacidade 6:

$$(5 + 1) \% 6 = 0$$

Depois da última posição vem a primeira, como num relógio.

enqueue e dequeue: $O(1)$

Onde encontramos filas

1

Impressora

Os documentos são impressos por ordem de envio.

2

Servidores

Pedidos de utilizadores atendidos por ordem de chegada.

3

Sistema operativo

Processos à espera de CPU ou de disco.

4

Pesquisa em largura

Percorrer um grafo "por camadas" (próximas aulas).

Lembra-se de Huffman (Aula 6)? Usava uma fila de prioridade: sai o elemento mais "leve", não o mais antigo.

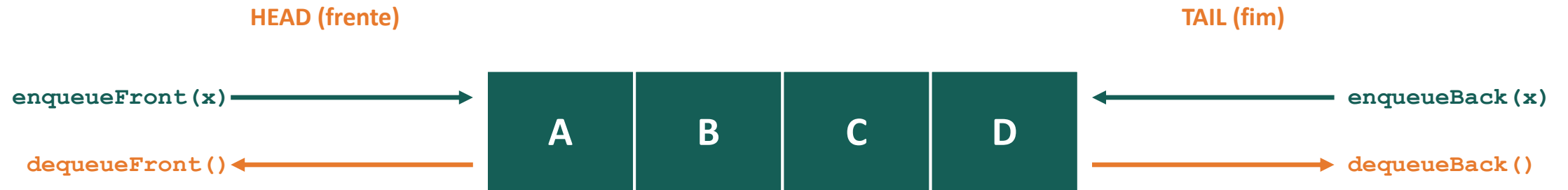
3

PARTE 3

Filas duplas

Entrar e sair pelas duas pontas.

Uma fila com duas portas



Como pilha

Usar só uma ponta: enqueueFront + dequeueFront.

Como fila

Entrar por uma ponta e sair pela outra: enqueueBack + dequeueFront.

As duas coisas

Ex.: histórico com limite — junta à frente e descarta o mais antigo atrás.

Entradas e saídas nas duas pontas

início

vazia

enqueueBack (A)



enqueueBack (B)



enqueueFront (C)



dequeueBack () → B



enqueueFront (D)



dequeueFront () → D



Custo

As quatro operações são $O(1)$

com um array circular ou com uma lista duplamente ligada.

As classes que o Java já oferece

Estrutura	Declaração	Entrar	Sair	Espreitar
Pilha	<code>Deque<Integer> p = new ArrayDeque<>();</code>	<code>push(x)</code>	<code>pop()</code>	<code>peek()</code>
Fila	<code>Queue<Integer> f = new ArrayDeque<>();</code>	<code>offer(x)</code>	<code>poll()</code>	<code>peek()</code>
Fila dupla	<code>Deque<Integer> d = new ArrayDeque<>();</code>	<code>offerFirst(x)</code> <code>offerLast(x)</code>	<code>pollFirst()</code> <code>pollLast()</code>	<code>peekFirst()</code> <code>peekLast()</code>

Dica

Prefira `ArrayDeque` à classe `Stack` (antiga). Também pode usar `LinkedList`, que implementa `Deque`.

Mas primeiro, faça à mão

Nos exercícios 1 e 3 vai implementar as suas próprias classes com um array: é assim que se percebe o que está por dentro.

Pilha, fila e fila dupla

	Pilha	Fila	Fila dupla
Regra	LIFO	FIFO	as duas pontas
Entra por	topo	fim (tail)	frente ou fim
Sai por	topo	frente (head)	frente ou fim
Sai primeiro	o mais recente	o mais antigo	depende da ponta
Analogia	pilha de pratos	fila da cantina	baralho de cartas
Operações	O(1)	O(1)	O(1)

Agora é consigo

1 Pilha com array

Classe pilha de inteiros com `push()`, `pop()` e `size()`, sobre um array de capacidade 1000.

Trate a pilha cheia e a pilha vazia.

3 Fila com array

Classe fila de inteiros com `enqueue()`, `dequeue()` e `size()`, sobre um array de capacidade 1000.

Pista: array circular.

2 Inverter n elementos

`reverse(p, 3)` sobre `{1,2,3,4,5}` (1 no topo) dá `{3,2,1,4,5}`.

Pista: uma fila auxiliar ajuda.

4 Juntar filas ordenadas

Fundir duas filas crescentes numa nova: `[2,4,8,10] + [1,4,9] → [1,2,4,4,8,9,10]`.

É o Merge da Aula 4!

Variações sobre o mesmo tema

5

Fila ordenada

Variante do exercício 3 em que os elementos ficam guardados por ordem crescente.

*Onde é que o enqueue tem de inserir?
Qual passa a ser o seu custo?*

6

Fila dupla

Variante do exercício 3 com filas duplas: enqueueFront, enqueueBack, dequeueFront, dequeueBack.

O array circular continua a servir.

7

Com LinkedList

Novas versões dos exercícios 1 e 3 usando uma lista duplamente ligada (LinkedList do Java).

O que ganha? E o que perde?

Três ideias a reter

1

Pilha = LIFO

push e pop no topo. Anular, chamadas de funções, parênteses.

2

Fila = FIFO

Entra atrás, sai à frente. Com array circular, tudo em $O(1)$.

3

Fila dupla

Entra e sai pelas duas pontas; pode ser pilha ou fila. Em Java: ArrayDeque.

A regra de acesso é tudo: muda a regra e muda a estrutura.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.^a ed.). Pearson.

2

Areias, M. (2023/24). *Estruturas de Dados — Aula prática 7*. DCC-FCUP. dcc.fc.up.pt/~miguel-areias/teaching/2324/ed/praticas/aula07.html

Diagramas redesenados a partir dos materiais de M. Areias.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt